

ステップ5: タスク内容の編集機能と保存 (発展)

前のステップでは、タスクの追加、表示、完了・削除機能の永続化を学びました。この発展ステップでは、既存のタスク内容を編集し、変更をlocalStorageに保存する方法を学びます。

1. このステップのゴール

- タスクの「編集」ボタンを追加し、既存のタスクテキストを変更できるようにする。
- 編集後の内容を `localStorage` に保存し、ページ再読み込み後も反映を維持する。

2. 事前知識の確認

- JavaScriptの `prompt` 関数またはインライン編集で文字列を取得する方法。
- 既存の `tasks` 配列の要素に変更を加え、`saveTasks()` と `renderTasks()` を呼び出す流れ。

3. 具体的な手順: `script.js` の変更

3.1. `editTask` 関数の作成

```
// タスクを編集する関数
function editTask(id) {
  const task = tasks.find(t => t.id === id);
  if (!task) return;
  const newText = prompt('タスク内容を編集してください', task.text);
  if (newText !== null && newText.trim() !== '') {
    task.text = newText.trim();
    saveTasks();      // 変更をlocalStorageに保存
    renderTasks();    // 画面を再描画
  }
}
```

3.2. `renderTasks` の更新

- 各タスクに「編集」ボタンを追加し、クリック時に `editTask` を呼び出します。

```
// renderTasks 内のボタン設定箇所
const editButton = document.createElement('button');
editButton.textContent = '編集';
editButton.classList.add('edit-btn');
editButton.addEventListener('click', () => editTask(task.id));
// append順: 完了ボタン, 編集ボタン, 削除ボタン
li.appendChild(completeButton);
li.appendChild(editButton);
li.appendChild(deleteButton);
```

3.3. データフローイメージ

```
flowchart LR
    tasks(タスク配列) -->|JSON.stringify| storageStr(JSON文字列)
    storageStr -->|localStorage.setItem| localStorage
    localStorage -->|localStorage.getItem| loadedStr(JSON文字列)
    loadedStr -->|JSON.parse| loadedTasks(タスク配列)
    loadedTasks -->|renderTasks| 画面表示
    renderTasks -->|編集ボタン| editTask
```

※Mermaid図を表示するには、VS Code の `Markdown Preview Mermaid Support` などの拡張機能が必要です。

3.4. インライン編集サンプル

```
// タスクテキストを直接編集するサンプル
function enableInlineEdit() {
    document.querySelectorAll('.task-text').forEach(span => {
        span.contentEditable = true;
        span.addEventListener('blur', () => {
            const id = Number(span.dataset.id);
            const task = tasks.find(t => t.id === id);
            const newText = span.textContent.trim();
            if (!task) return;
            if (newText) {
                task.text = newText;
                saveTasks();
                renderTasks();
            } else {
                alert('タスク内容を空にできません。');
            }
        });
    });
}
```

```
renderTasks();  
}  
});  
});  
}  
  
// renderTasks() の後に呼び出して有効化  
// enableInlineEdit();
```

4. 演習課題

1. 演習用ベースコード の `script.js` に上記「3.1. `editTask` 関数の作成」と「3.2. `renderTasks` の更新」を実装してください。
2. 解答例/step5_edit フォルダのコードを参考に、タスクの編集機能が動作することを確認してください。

5. 動作確認の方法

1. タスクをいくつか登録する。
2. 各タスクに表示された「編集」ボタンをクリックし、テキストを変更する。
3. 変更後、リロードしても編集内容が保持されていることを確認する。

テスト観点 (チェックリスト)

- 編集キャンセル時に元の内容が保持されるか
- 空入力時に適切なアラートが出るか
- 連続編集で正しく最新の内容が反映されるか
- 他の機能（完了・削除）と干渉がないか

6. Q&A

Q1: `prompt` を使わず、インラインで編集したい場合は？ A1: `contenteditable` 属性を使っ

て `span` 要素を直接編集可能にし、編集完了時にイベントリスナーで変更を検知し、
`saveTasks()` と `renderTasks()` を呼び出す方法もあります。演習で挑戦してみましょう。

Q2: 編集キャンセル時の挙動はどうなりますか？ A2: `prompt` でキャンセルすると

`newText` が `null` になるため、変更処理を実行せずそのまま終了します。

```
// エラーハンドリング付き editTask の例
function editTask(id) {
  const task = tasks.find(t => t.id === id);
  if (!task) {
    console.error(`タスクが見つかりません: ${id}`);
    return;
  }
  const newText = prompt('タスク内容を編集してください', task.text);
  if (newText === null) {
    // キャンセル時は何もしない
    return;
  }
  const trimmed = newText.trim();
  if (trimmed === '') {
    alert('タスク内容を空にすることはできません。');
    return;
  }
  task.text = trimmed;
  saveTasks();
  renderTasks();
}
```

7. よくあるエラーと対処法

- `prompt` ダイアログが表示されない → ブラウザのポップアップブロック設定を確認する。
- 空文字で確定するとアラートが出ることを確認する。
- 編集後に内容が保存されない → `saveTasks()` が呼ばれているかコンソールでエラーを確認する。
- タスクIDが見つからない → 一意な `id` が付与されているか `tasks` 配列の構造を確認する。
- Mermaid 図が表示されない → VS Code の拡張機能（Markdown Preview Mermaid Supportなど）をインストールする。